Python Programming Foundations

Week 1 - First Programs and Basic Values

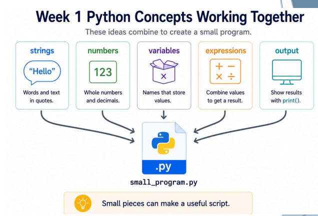
Day 3: "Basic Values Together"

1

Put The Pieces Together

This week, you learned the smallest useful Python pieces.

Today, you combine them into tiny programs that run and make sense.



2

Quick Review

Monday made output visible.

Tuesday made numeric value flow visible.

3

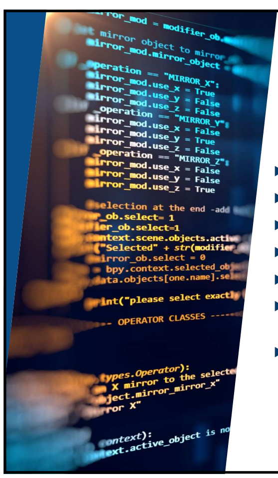
What Counts as Success Today

NOT a large program

Small Program that:

- Runs
- Produces output
- Can be Explained

4



What We Will Use Today

- strings
- numbers
- variables
- simple expressions
- `print()`
- optional `input()` if stable
- basic type awareness

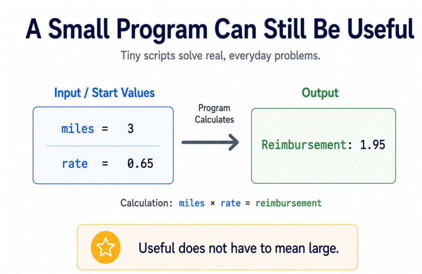
5

Working Set

6

What to “Skip” for Now

7



A useful first program does not have to be large.

It only needs a clear purpose, visible output, and values you can explain.

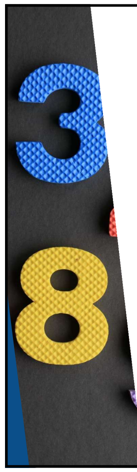
8

Combine Values Carefully

Your program may use text and numbers together.

The important question is whether you can explain each value's role.

9

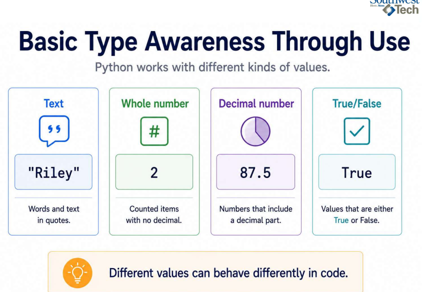


Type Awareness Through Use

For Week 1, recognize:

- Text
- Whole numbers
- Decimal numbers
- True/false values (boolean)

10



Type Awareness

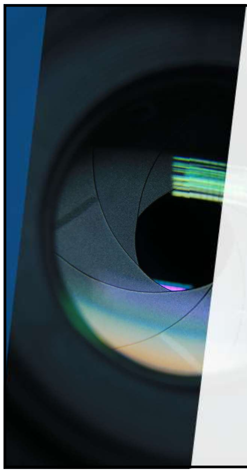
11

Explain Every Main Line

Before submitting Assignment 1, ask: can I explain every main line?

If a line works but you cannot explain it, slow down and inspect it.

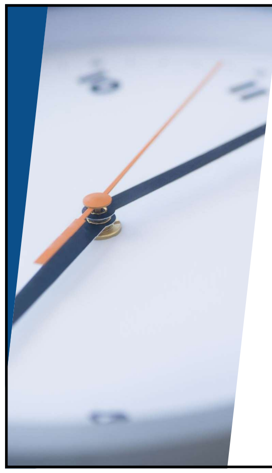
12



Demo #1

"Basic Type Awareness"

13



Southwest Tech

What to watch for

As the demo runs, watch for two things:

- How the program stores several different kinds of values.
- Recognition. This is the goal - ***not*** a full type inventory.

14

Southwest Tech

Basic Type Awareness Demo

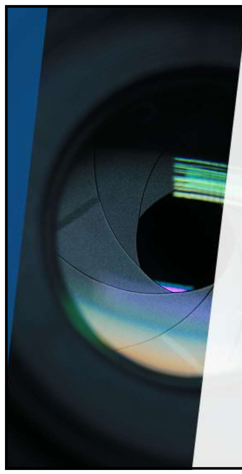
Every variable holds a value, and each value has a kind.

Variable Name	Value	Kind of Value
student_name	"Riley"	text
assignments_completed	2	whole number
average_score	87.5	decimal number
is_passing	True	true/false

Recognize the value before you explain the program.

Type Demo

15



Optional Demo #2

"Input And Number Conversion"

16

Southwest Tech

Optional Demo: Input And Number Conversion

- When users type into `input()`, Python receives text first.
- For math, the text must be converted into a number.

17

Southwest Tech

Typing vs. Copy-Paste

18

Southwest Tech

Learning Move:
Type It When It Is New

Typing code can be part of learning.

Copying is not automatically wrong, but it should not replace understanding.

19

Southwest Tech

Typing it Out

Typing code manually slows the process down in a useful way.

When you type an example, you are not just moving characters onto the screen. You are giving your mind time to process:

- the order of the instructions
- the names of variables
- the structure of the logic
- the punctuation and syntax
- the relationship between one line and the next
- what the program is actually doing

20

Southwest Tech

Copying and Pasting

Copying and pasting can also be appropriate. It may make sense when:

- the code is long
- the concept has already been learned
- the focus is on a different part of the task
- you understands the logic already
- the code is being used as a scaffold for something else

Copy/paste becomes a problem when it replaces understanding. - Before copying, You should ask:

- Do I understand what this code does?
- Do I need to understand this more deeply right now?
- Am I copying because it saves time, or because I am avoiding thinking?

21

Southwest Tech

Connection to AI Prompting

Typing a prompt is not just the act of entering text into a tool. It is often part of the thinking process.

As a person writes a prompt, they may:

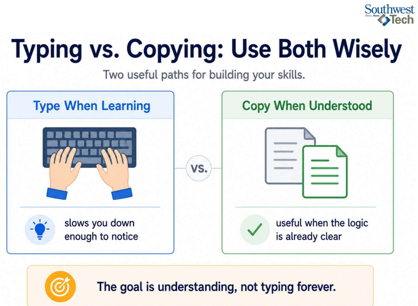
- clarify the goal
- revise wording
- discover missing constraints
- notice ambiguity
- reshape the request
- improve the structure of their own thinking

This is closely connected to AI-assisted development and Refraction-Based Architecture.

- The human is not merely asking AI for output - ***The human is shaping intent!***

22

Southwest Tech



Typing vs Copy-Pasting

23

Common Failure:
Too Much Is Too Much

Trying to impress can make the first assignment harder than it needs to be.

Small, clean, and explainable is better than large and confusing.

24

Common Failure: Working But Unexplained

- Code that runs is not the whole goal.
- You also need to explain what values the program uses and what output it creates.

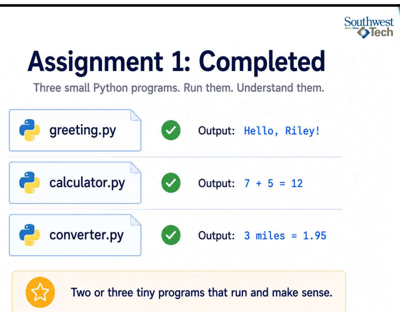
25

From Demo to Lab

For Assignment 1:

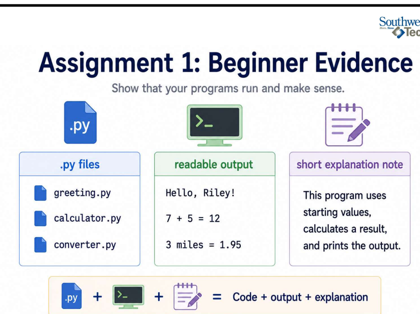
- Finish two or three tiny programs.
- Each program should run, show output, and make sense when you explain it.

26



Assignment Completion

27



Assignment Evidence

28

Lab On Success Check

- Week 1 worked if your small programs run and you can explain them.
- Understanding what the code is doing:
  - What values does (each) program use?
  - What output does it produce?
  - Where does a value change?
  - Could you explain this script line by line to someone else?

29

Thank you!

Have Fun!

30